

Alpacaruns — Hackathon One-Pager

Stack: Go 1.26, Alpaca Trading API (paper), Google ADK multi-agent framework, official uvx alpaca-mcp-server (separate data + exec toolsets), all trades routed through paper-api.alpaca.markets/v2. Single static Go binary, distroless Docker image, systemd unit, JSONL trade log on every fill.

1. AI logic — Mixture-of-Experts ensemble

Two cooperating inference paths run side by side:

LLM path (agents/agents.go, agents/mcp.go). A Google-ADK graph with a gating root that routes a query to either the trading cycle or the monitor loop. The cycle runs `MarketData` -> `AnalysisParallel` (`Technical` || `Sentiment`) -> `TradeIdea` -> `RiskManagementExpert` -> `ExecutionExpert`. The `ExecutionExpert` calls Alpaca through the official MCP server (uvx alpaca-mcp-server); every order tool is wrapped in `guardedTool` (agents/mcp.go) so the deterministic risk gate runs **before** the MCP tool reaches the wire. Model selection is pluggable: `config.LLMProvider` switches between a local llama.cpp OpenAI-compatible endpoint, Gemini, and Oxlo.ai — no code change between them.

Deterministic path (strategy/, factors/). Five-factor composite (trend 0.30, momentum 0.25, volume 0.20, volatility 0.15, sentiment 0.10) feeds a Layer-2 ensemble of six expert voices: **trend**, **mean-rev**, **breakout**, **pairs** (e.g. SPY/QQQ, XLE/XLF), **xsmom** (cross-sectional momentum), and **seasonality** (strategy/ensemble/). Each tick fetches bars **once** via `runner.BuildMarketData`, shares them read-only across every expert, then the gater (strategy/ensemble/gater.go) stacks the votes with performance-weighted mass — $\text{weight} = \text{baseWeight} \times (0.5 + \text{hitRate})$ — and applies a vol-regime modifier (contrarian voices down-weighted in RisingVol/Crisis, trend voices up). A vote-variance breaker (`VarianceBreaker = 0.15`) forces **Hold** when the experts disagree too loudly; the winning mass must clear `MIN_ENSEMBLE_CONFIDENCE` (0.55) or the gater also forces Hold. Resolved outcomes feed a `perf.Window=30` hit-rate tracker persisted to `data/ensemble-state.json`, so weights adapt to live performance without retraining.

2. Risk gates

`risk/risk.go Validator.Validate` runs on **every** order from **every** path — agent execution, deterministic auto, and manual trade — and is **fail closed**: any error fetching live state, prices, or factor scores rejects the order, never passes it.

- **Per-position notional cap:** $\text{notional} \leq \text{MAX_POSITION_USD} (\text{equities}) / \text{CRYPTO_MAX_POSITION_USD} (\text{crypto})$. Options: $\text{qty} \times 100 \times \text{mark}$, `mark` = snapshot mid / last trade (price-fetch failure rejects).
- **Per-symbol cap** (`cmd/alpacaruns/auto.go:880-895`): ``existing + notional ≤ MaxPositionUSD; crypto gets 3× CRYPTO_NOTIONAL_USD`` to allow scale-in before the cap kicks in.
- **Portfolio percentage cap:** $\text{notional} / \text{equity} \leq \text{MAX_PORTFOLIO_PCT}$ (additive: existing position + new order).
- **Confidence + factor gate:** $\text{Confidence} \geq \text{MIN_CONFIDENCE}$ AND $\text{composite} \geq \text{FACTOR_MIN_SCORE}$; per-factor rationales named in every rejection.

- **Drawdown halts:** daily $\geq$ DAILY_DD_HALT (5%), weekly $\geq$ WEEKLY_DD_HALT (10%), total $\geq$ TOTAL_DD_HALT (15%) of peak equity engage the shared kill switch — `agents.KillSwitch.Halted()` is the very first gate every validator consults, and nothing further trades until restart.
- **Session rules:** extended-hours allowed only for ``type=limit,`
`time_in_force=day|gtc``, weekdays 04:00–20:00 ET; options and market orders always fail closed outside regular hours; crypto trades 24/7.
- **Options constraints** mirrored client-side from Alpaca's server: whole contracts, no notional, no extended hours, `tif=day|gtc`.

3. Alpaca infrastructure

Paper endpoint: `https://paper-api.alpaca.markets/v2`. A non-paper URL is refused by `config/config.go` unless `I_ALPACA_LIVE=YES` is set — paper is the hard default. The Go client lives in `tools/alpaca.go` (typed REST: orders, bars, quotes, snapshots, news, account, positions) with a sibling `options/options.go` for OCC contracts/snapshots/greeks.

The MCP server is spawned by `agents/mcp.go: exec.Command("uvx", "alpaca-mcp-server")` with `ALPACA_API_KEY/ALPACA_SECRET_KEY` injected as env, `transport mcp.CommandTransport`. Both data and exec toolsets are registered. **Decision** → **order path:** `GatingRoot` → `ExecutionExpert` → `guardedTool (validates)` → `mcptoolset.ProcessRequest` → Alpaca MCP server → `/v2/orders`. The deterministic auto loop bypasses MCP and calls the typed client directly; both paths converge on the same `risk.Validator`. Every fill arrives on the `trade_updates` stream and is journaled to `data/trades.jsonl` in real time, deduplicated by Alpaca order ID; `pl [--since DATE]` backfills fills missed while the process was down.

4. Honest limitations

- The **options overlay degrades to equity on free tier** — the live journal shows no tradable ca... because `/v2/options/snapshots` 404s without a paid data plan. `OPTIONS_ENABLED` defaults to true, so on free plans the bot logs a skip and falls through to the equity path (`strategy/options_overlay.go`).
- The pre-hackathon account PA3LNUDV231J was created **2026-08-25**, three days before the 2026-08-28 threshold and ineligible. A fresh post-2026-08-28 paper account replaces it; the bot is being restarted against the new keys and P&L below is from a fresh series.
- All P&L is paper-only; no live capital has been traded.
- Local-LLM tool calling depends on model and chat-template setup (`--jinja` for Qwen3) — malformed tool calls remain the top local-model failure mode.
- No backtesting engine; thresholds validated by unit tests and live paper runs.